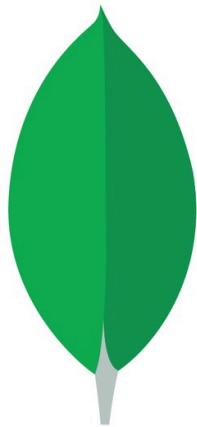


UT4. Proyecto ABP- Acceso a BD Clinica



mongoDB®

Adriá Hortonedá Martínez
Víctor Samuel González González
Alexander Fuentes Rodríguez

Descripción breve del proyecto.....	3
Estructura de carpetas.....	3
Instalación.....	4
Descripción de las dependencias.....	4
Uso.....	5
Nota importante.....	5
Tecnologías.....	5

Descripción breve del proyecto

Aplicación desarrollada con Flet para la gestión integral de citas en una clínica médica. El sistema permite la inserción, modificación y eliminación de datos de pacientes, médicos y consultas. Gracias a la tecnología Flet, la aplicación es compatible con dispositivos móviles, escritorio y web, ofreciendo una experiencia de usuario consistente en todas las plataformas.

Características principales:

- Gestión completa de pacientes, médicos y consultas
- Operaciones CRUD (Crear, Leer, Actualizar, Eliminar) para todos los datos
- Interfaz intuitiva y fácil de usar
- Accesible desde múltiples dispositivos

Estructura de carpetas

```
trabajo_victor_alex_adria/  
├── Proyecto/  
│   ├── models/  
│   │   ├── __init__.py  
│   │   └── db_models.py  
│   ├── base de datos  
│   │   ├──  
│   │   └── services/  
│   ├── CRUD  
│   │   ├── __init__.py  
│   │   └── crud_operations.py  
│   ├── operaciones CRUD  
│   │   └── GeneracionDatos.py  
│   ├── prueba  
│   │   └── mongo_service.py  
│   └── MongoDB  
│       ├──  
│       ├── utils/  
│       │   ├── __init__.py  
│       │   ├── labels.py  
│       │   ├── style.py  
│       │   └── validators.py
```

Directorio para modelos de datos

Hace de 'models' un paquete

Modelos de colecciones de la base de datos

Directorio para operaciones CRUD

Hace de 'services' un paquete

Implementación de operaciones CRUD

Generación de datos de prueba

Servicios de conexión con MongoDB

Directorio de utilidades

Hace de 'utils' un paquete

Mapeo de etiquetas

Estilos y temas visuales

Validaciones de datos

```

|   |
|   |   └─ views/                                # Directorio de vistas
|   |       └─ __init__.py                        # Hace de 'views' un paquete
|   |       └─ alert_view.py                      # Vista de alertas
|   |       └─ delete_view.py                    # Vista de eliminación
|   |       └─ inserciones_view.py               # Vista de inserción
|   |       └─ leer_views.py                     # Vista de lectura
|   |       └─ main_view.py                      # Vista principal
|   |       └─ update_views.py                   # Vista de actualización
|   |
|   └─ app.py                                    # Archivo principal para iniciar la
aplicación
|   └─ main.py                                  # Archivo secundario para inicio
en Linux

```

Instalación

Para la correcta funcionalidad del programa, siga estos pasos desde una terminal:

1. Crear un entorno virtual:

python -m venv [nombre_del_entorno]

2. Activar el entorno virtual:

[nombre_del_entorno]\Scripts\activate.bat

3. Instalar las dependencias necesarias:

pip install flet pymongo faker

Descripción de las dependencias

- flet: Framework para crear interfaces de usuario adaptables a móviles y web.
- pymongo: Librería para conectar y realizar operaciones CRUD en bases de datos MongoDB.
- faker: Generador de datos aleatorios útil para pruebas y desarrollo.

Uso

1. Ejecuta el programa.
2. Al iniciar, seleccione la opción "Generar datos". Esta acción es crucial ya que:
 - Crea un conjunto de datos aleatorios.
 - Establece la base de datos local necesaria para el funcionamiento del programa.
3. Una vez generados los datos iniciales, podrá acceder y utilizar todas las funcionalidades del programa con normalidad.

Nota importante

La generación de datos solo es necesaria la primera vez que se ejecuta el programa o si se desea añadir más datos en la base de datos. En ejecuciones posteriores, puede omitir este paso si ya cuenta con datos en su base local.

Tecnologías

Python

- Lenguaje de programación principal del proyecto
- Implementado con una arquitectura modular y orientada a objetos
- Organizado en paquetes (models, services, utils, views)
- Gestión de dependencias mediante entorno virtual

MongoDB

- Sistema de base de datos NoSQL
- Integración mediante pymongo
- Implementación de operaciones CRUD en crud_operations.py
- Servicio de conexión gestionado por mongo_service.py

Flet

- Framework para desarrollo de interfaces gráficas
- Estructura de vistas organizada en el directorio views/
- Componentes principales:
 - alert_view.py para notificaciones
 - inserciones_view.py para crear registros
 - leer_views.py para consultas

- `update_views.py` para modificaciones
- `delete_view.py` para eliminación
- `main_view.py` como vista principal